Estructuras de Árboles B

Conceptos fundamentales sobre organización de archivos, árboles:

- **A-B**
- **B**
- **B+**
- **B***

Anexo:

- **Áboles lexicográficos (TRIE y Patricia).**

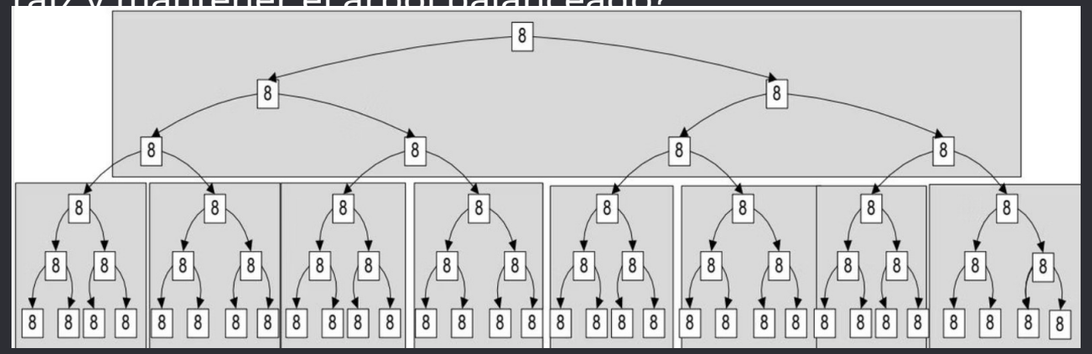
por Carlos Alberto Rico

Valeria A. Di Croce

Árboles Binarios Paginados y Motivación de los Árboles B

Árboles Binarios Paginados

Abordan problemas de almacenamiento secundario mediante buffering y páginas de memoria con varios registros. Minimizan el número de accesos. El principal problema es la construcción descendente: ¿cómo elegir la raíz y mantener el árbol balanceado?



Motivación: Tiempo de Acceso

En almacenamiento masivo, el **tiempo de acceso** supera al **tiempo de transferencia**. Estructuras enlazadas como AVL acceden a muchos nodos pequeños. La solución: **reducir el número de accesos** usando nodos de **mayor tamaño**.

¿Por qué necesitamos árboles A-B?

Imaginemos tener millones de datos guardados en disco.

El problema no es solo buscar…
El problema es que **leer del disco es lento**

Entonces queremos:

- Hacer **la menor cantidad de accesos a disco posible**
- Reducir la **altura del árbol**
- Agrupar muchos datos en cada nodo

Problema con árboles binarios

Un árbol binario (como ABB o AVL):

- Tiene **máximo 2 hijos por nodo**
- Crece mucho en altura

Ejemplo:

- 1 millón de datos → altura grande → muchas lecturas de disco

Resultado: **ineficiente para bases de datos**

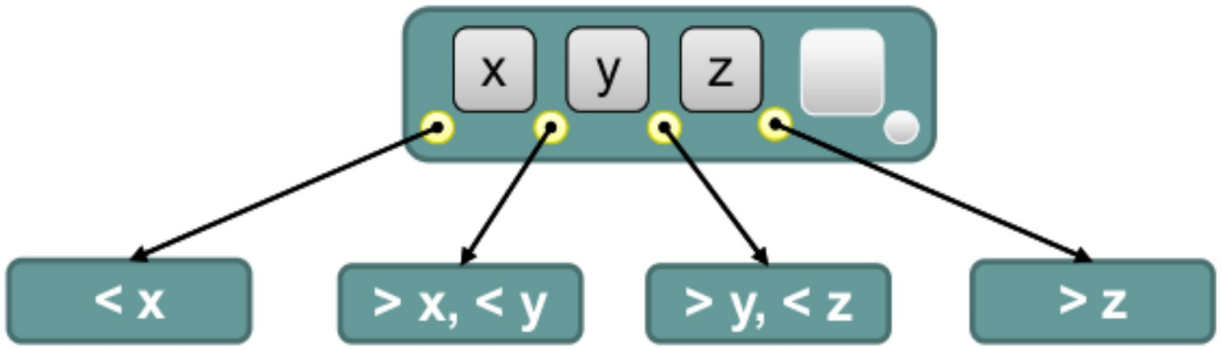
Solución: Árboles A-B

La idea clave es:

En vez de 2 hijos, tener MUCHOS hijos por nodo

Así:

- El árbol crece **a lo ancho**
- Y no tanto **a lo alto**



Definición de árbol A-B

Un árbol (A, B) es un árbol balanceado (no binario) donde:

- Cada nodo tiene entre **A y B hijos**
- Cada nodo tiene un número de hijos **k** tal que:

  $\mathbf{2 \leq A \leq k \leq B}$

- El **orden del árbol** es:

  m = B (máximo número de hijos)

- Todos los nodos hoja están al mismo nivel

Nota: en algunos autores **m** es el orden que seria **B**

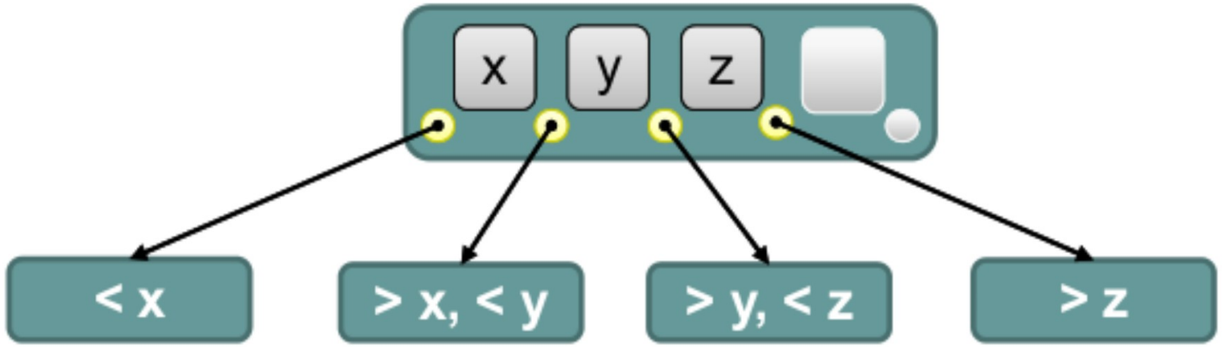
Propiedades importantes

1. **Número de hijos**
   - Mínimo: A
   - Máximo: B
2. **Número de claves**
   - Si tiene k hijos → tiene k-1 claves
3. **Orden interno**
   - Las claves dentro del nodo están **ordenadas**
4. **Balance**
   - Todas las hojas están a la misma profundidad

Contenido de un nodo

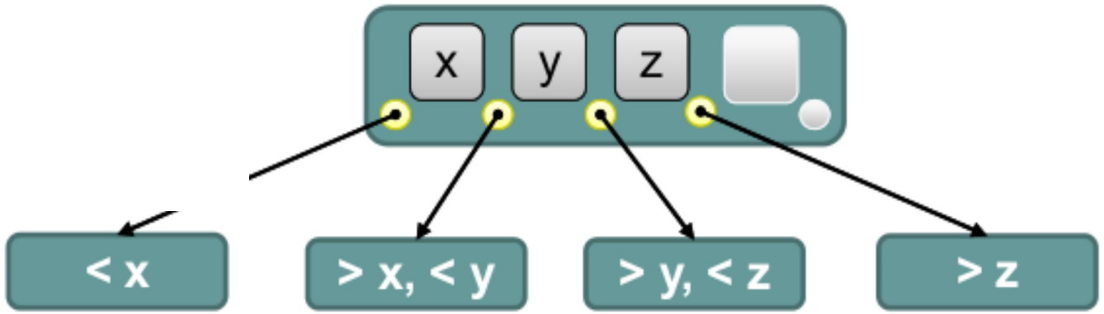
Cada nodo almacena:

- Un número **k** → cantidad de hijos del nodo
- Un arreglo de **claves ordenadas**



Árboles (A,B)

- Los **árboles (A,B)** son árboles generales (no binarios) donde cada nodo interno puede tener un número de hijos **k** en el rango **[A, B]**.
- Cada nodo almacena **k - 1 claves** (elementos comparables por ≤), ordenadas de menor a mayor, que sirven para que se pueda usar como un **árbol de búsqueda**.
- El contenido típico de un nodo consiste en:
  - Un entero **k**, con **A ≤ k ≤ B**, que indica el número de hijos del nodo.
  - Un vector **c**, de capacidad **B - 1**, que almacena las **k - 1 claves**.
  - Un vector **h**, de capacidad **B**, que almacena los **k enlaces a hijos**.
- **Propiedad de ordenación**:
  - `h[0]` almacena claves **menores que** `c[0]`
  - `h[1]` almacena claves **mayores que** `c[0]` y **menores que** `c[1]`
  - `h[2]` almacena claves **mayores que** `c[1]` y **menores que** `c[2]`
  - ...
  - `h[k-1]` almacena claves **mayores que** `c[k-2]`



Árbol B de Orden m

Árbol B (Bayer y McCreight, 1972)

El **árbol B** fue introducido por:

> **Rudolf Bayer y Edward McCreight en 1972**

Un **árbol B** es básicamente:

Un caso particular de un árbol (A,B)

A y B no son arbitrarios, sino que están definidos en función del **orden m**.

Definición formal (conectada a A-B)

Un árbol B de orden **m** es un árbol (A,B) donde:

- **B = m** → máximo de hijos
- **A = ⌈m/2⌉** → mínimo de hijos

¿Por qué se fija A así?

Para garantizar que:

- El árbol esté **balanceado**
- No haya nodos "casi vacíos"
- La altura sea **mínima posible**

Esto es CLAVE para bases de datos

Propiedades del Árbol B

Usando nuestra notación:

Para cada nodo:

- Tiene **k hijos**, con:

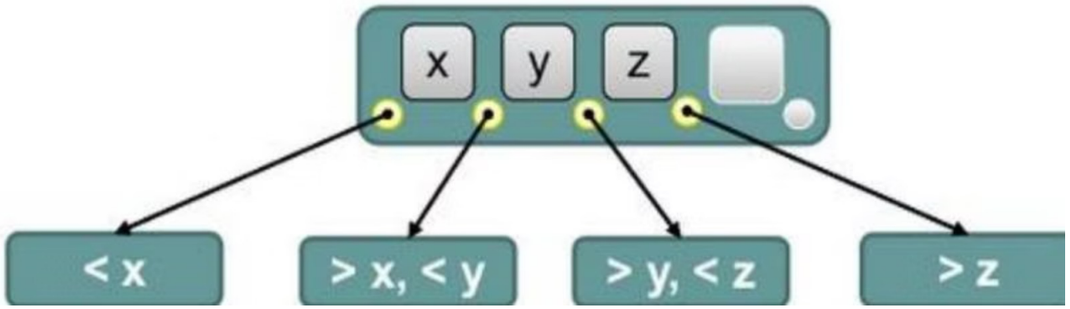
  $\lceil m/2 \rceil \leq k \leq m$

- Tiene **k - 1 claves**

Excepciones

La **raíz** puede:

- Tener menos hijos (incluso 2)
- O ser hoja



Árbol B de Orden m

Ejemplo concreto

Supongamos:

Árbol B de orden **m = 4**

Entonces:

- **B = 4**
- **A = ⌈4/2⌉ = 2**

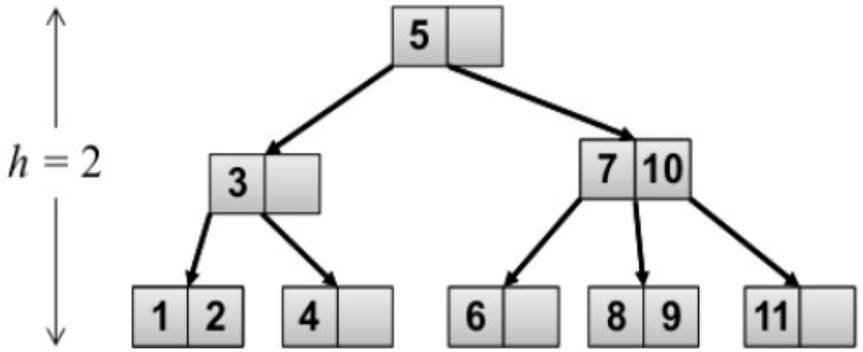
Es un árbol (2,4)

Ejemplo concreto

Árbol **B** de orden **m** = 3

A = ⌈m/2⌉ = 2

- Es un árbol (2,3)
- Propiedades:
  - $2 \leq k \leq 3$ hijos por nodo
  - $1 \leq$ claves ≤ 2
  - Altura h = 2
  - Todas las hojas al mismo nivel



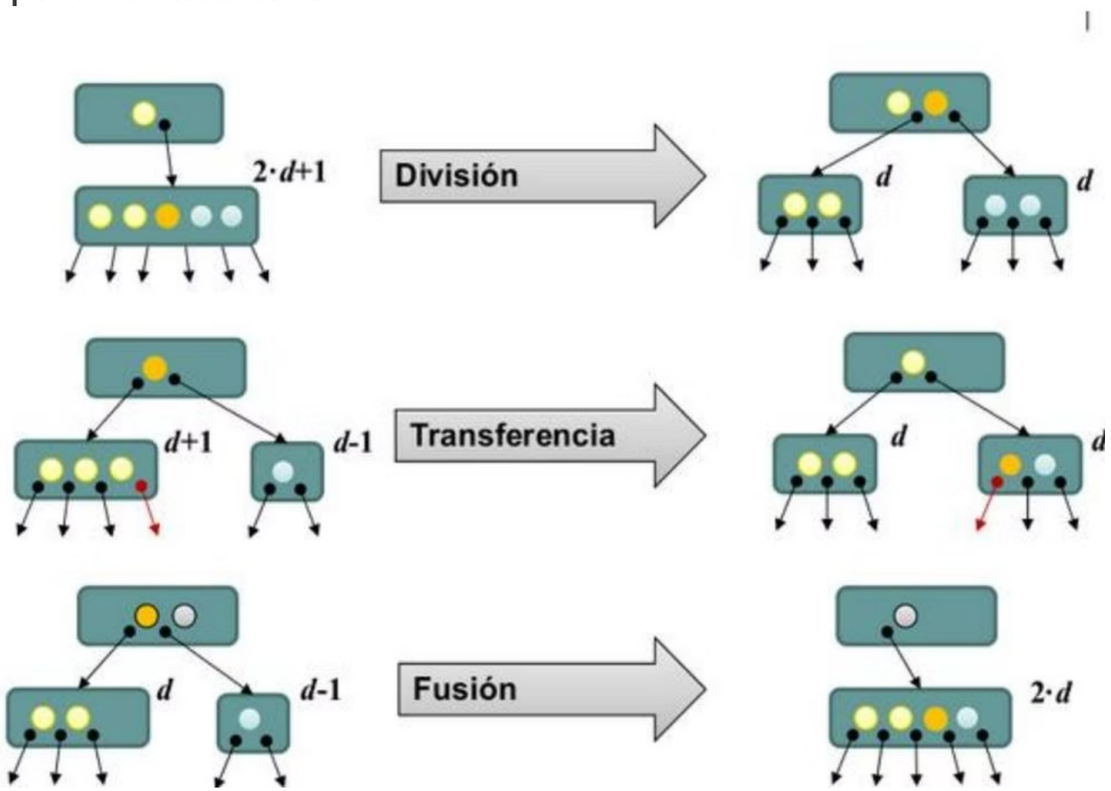
Comparación directa A-B vs B

Concepto	Árbol A-B	Árbol B
Hijos	$A \leq k \leq B$	$\lceil m/2 \rceil \leq k \leq m$
Parámetros	A y B libres	definidos por m
Orden	B	m
Uso	Teórico	Práctico (BD, discos)
Claves	k-1	k-1

Reestructuraciones, Búsqueda e Inserción

Reestructuraciones

División: nodo con 2d+1 claves se divide en dos de d claves; el padre gana una clave. **Transferencia:** se transfiere una clave entre nodos adyacentes.
Fusión: dos nodos adyacentes se fusionan; el padre pierde una clave.



Búsqueda

Se desciende desde la raíz buscando en el array de claves de cada nodo (secuencial o binaria). Si no se encuentra, se pasa al hijo asociado a la primera clave mayor.

Inserción

Se desciende hasta la hoja adecuada y se inserta. Si se supera el máximo de claves (2d), el nodo se **divide**, transfiriendo la clave media al padre. El proceso puede propagarse hacia arriba.

Animación de operaciones con Árboles B

Conclusiones y Usos de los Árboles B

Conclusiones Clave

Los árboles B surgen para mantener **índices externos** en bases de datos. La "B" **no significa binario**, ni son árboles de búsqueda convencionales, ni necesariamente balanceados.

Usos

Gestores de bases de datos y sistemas de ficheros: **NTFS** (Windows), **HFS+** (Apple), **btrfs**, **Ext4** (Linux).

Variantes

Árboles B+: solo las hojas contienen datos; hojas enlazadas en lista doble.
Árboles B*: mínimo 2/3 de capacidad; fusión de 3 nodos en 2.

Árboles B+ Tree

Surge como una **evolución del B-Tree.**

Limitación del B-Tree

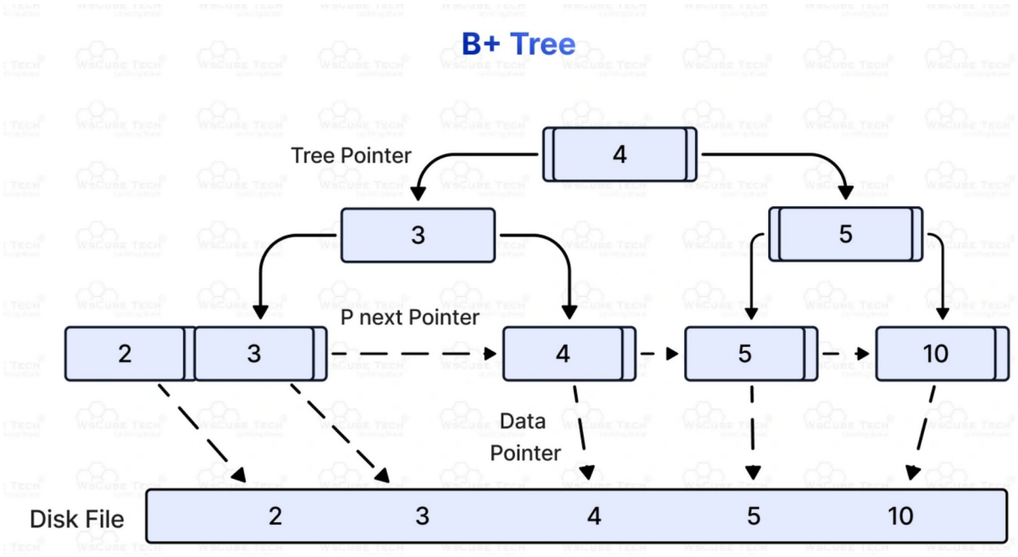
Los datos pueden estar:

- en nodos internos
- o en hojas

Problema:

- Recorrido secuencial menos eficiente
- Menor aprovechamiento de páginas

Solución: Evolución al árbol **B+**



Características del B+

- Nodos internos:
  - solo claves (índices)
- Hojas:
  - contienen **todos los datos**
- Hojas enlazadas entre sí

Importante: navegación (arriba) y datos (abajo)

Las hojas están enlazadas

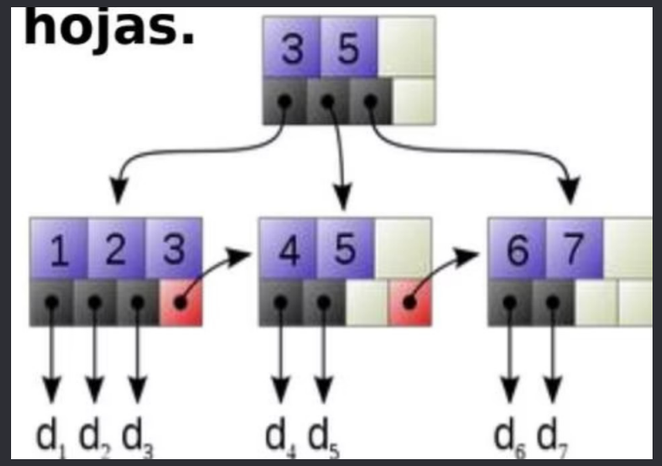
Ventajas:

- Altura muy baja
- Menos accesos a disco
- Mejor uso de páginas
- Permite:
  - recorridos secuenciales rápidos
  - consultas por rango eficientes

Árboles B+: Estructura y Propiedades

¿Qué es un Árbol B+?

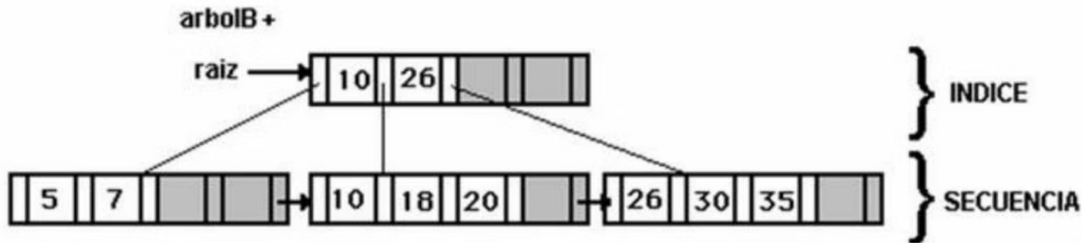
Índice multinivel, dinámico. **Toda la información** se guarda en las hojas. Los nodos internos solo contienen claves y punteros. Todas las hojas están al mismo nivel y **enlazadas como lista**, permitiendo recuperación en rango mediante búsqueda secuencial.



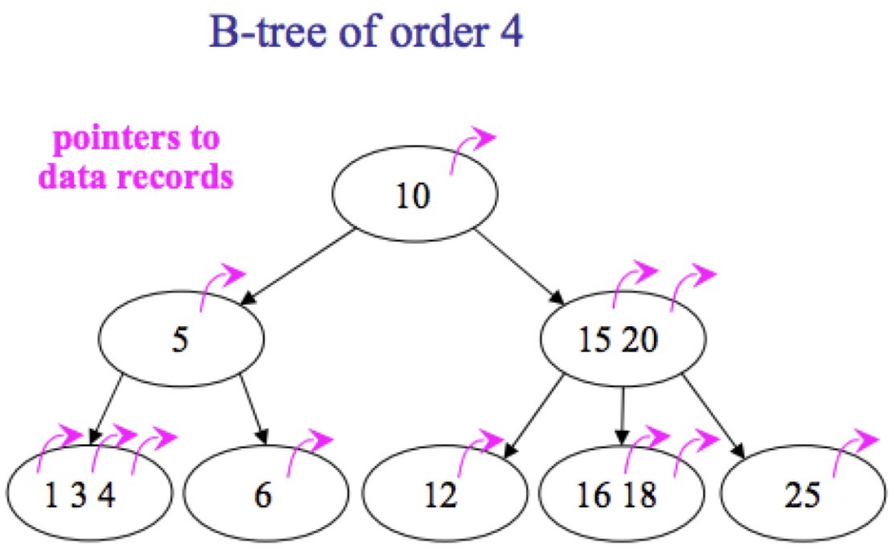
Propiedades

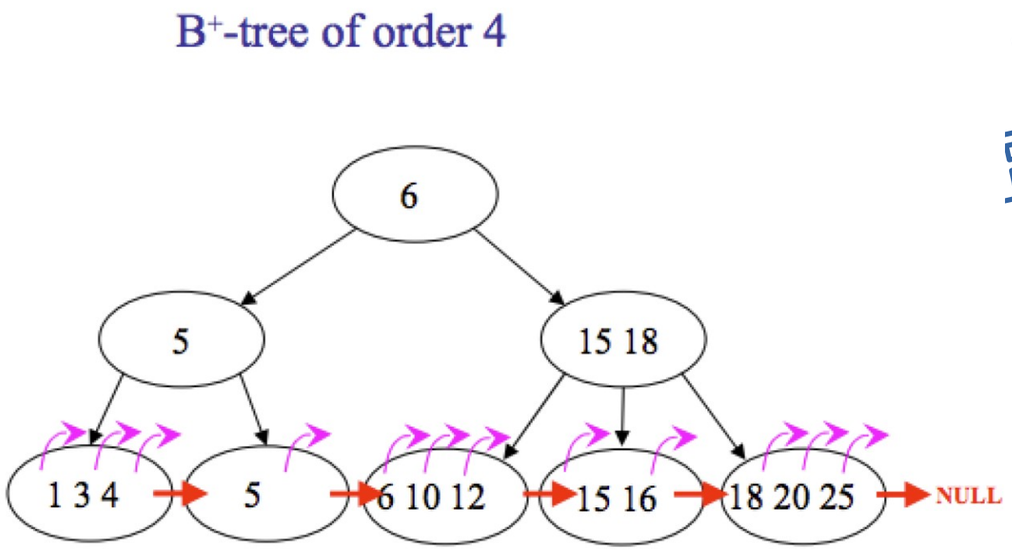
- Todas las claves en las hojas (duplicadas en nodos internos como índice).
- Hojas vinculadas → recorrido secuencial eficiente.
- Ocupa más espacio que B (claves repetidas).

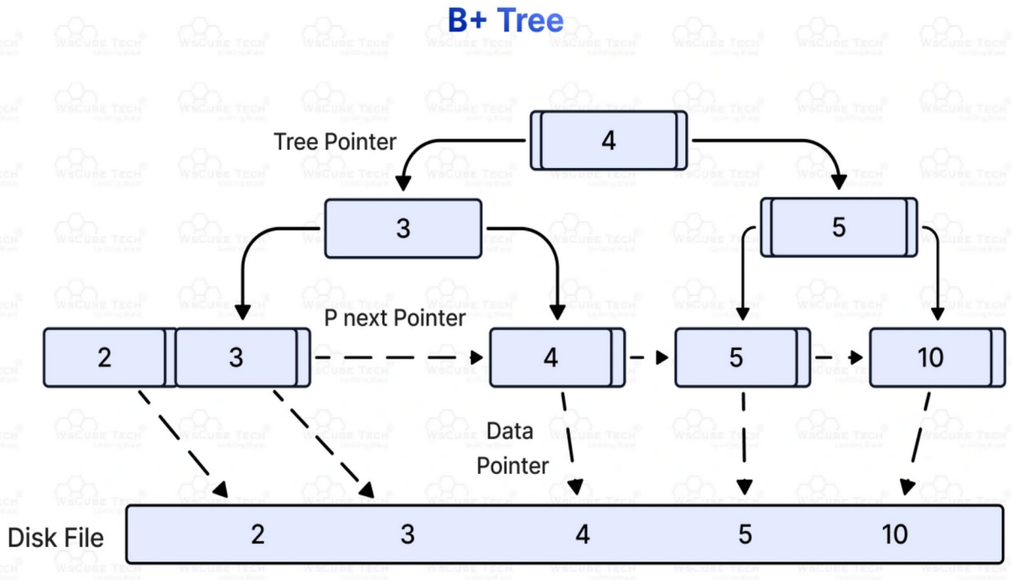
Compuesto por **Índice** (nodos interiores) y **Secuencia** (hojas enlazadas).



B vs B+







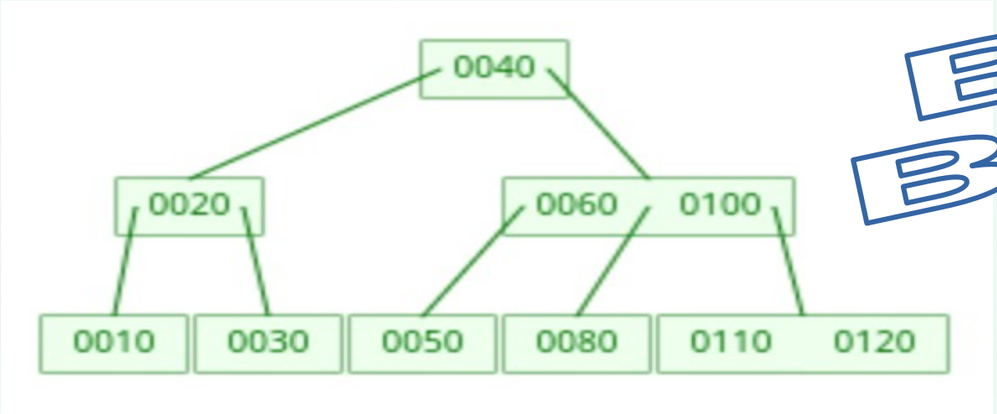
[Simulación Árbol B+]

Tabla Comparativa

Concepto	Árbol A-B	Árbol B	Árbol B+
Hijos	$A \le k \le B$	$\lceil m/2 \rceil \le k \le m$	$\lceil m/2 \rceil \le k \le m$
Parámetros	A y B libres	definidos por m	definidos por m
Orden	B	m	m
Uso	Teórico	Práctico (BD, discos)	Índices en BD, FS modernos
Claves	$k-1$	$k-1$	$k-1$ (internos)
Datos	—	Internos y hojas	Solo en hojas
Hojas enlazadas	—	**No**	**Sí**
Ocupación mínima	—	50%	50%
Estrategia	—	Split en 2	Split en 2

B vs. B+

B-Tree



- ✔ Datos en **nodos internos**
- ✘ Hojas no **enlazadas**
- ✔ Los datos pueden estar en interna y hojas
- ✔ 50% ocupación mínima

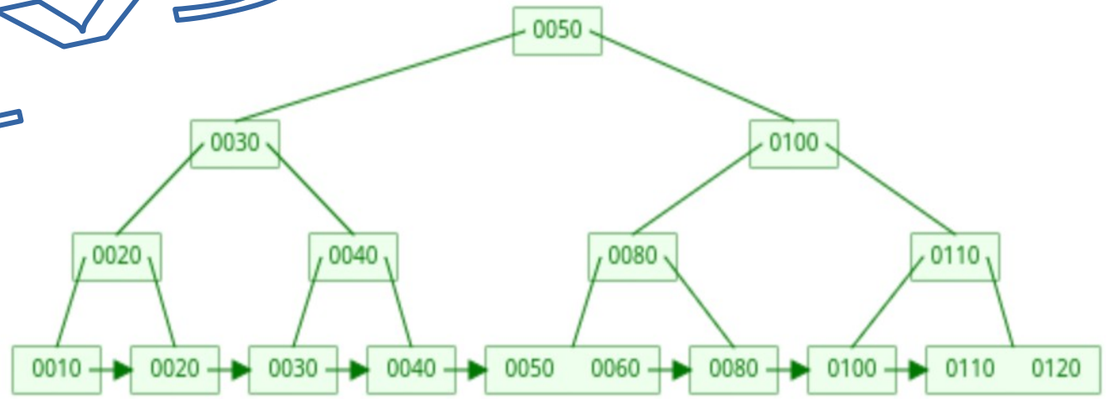
BD, discos antiguos

- ✔ BD, discos antiguos

- ✔ BD, discos **antiguos**

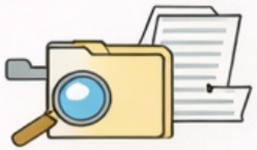
VS

B+ Tree



- ✔ Datos **solo** en hojas
- ✔ Hojas **enlazadas**
- ✔ Optimiza búsquedas por rango
- ✔ 50% ocupación mínima

MySQL PostgreSQL SQL Server



- ✔ BD, sistemas modernos

- ✔ BD, sistemas modernos

Árboles B* (Donald Knuth, 1973)

Definición:

Un **árbol B*** es una **variante del B-Tree** que

mejora el uso del espacio en los nodos.

Mantiene:

- Balanceo
- Multicamino
- Optimización para disco

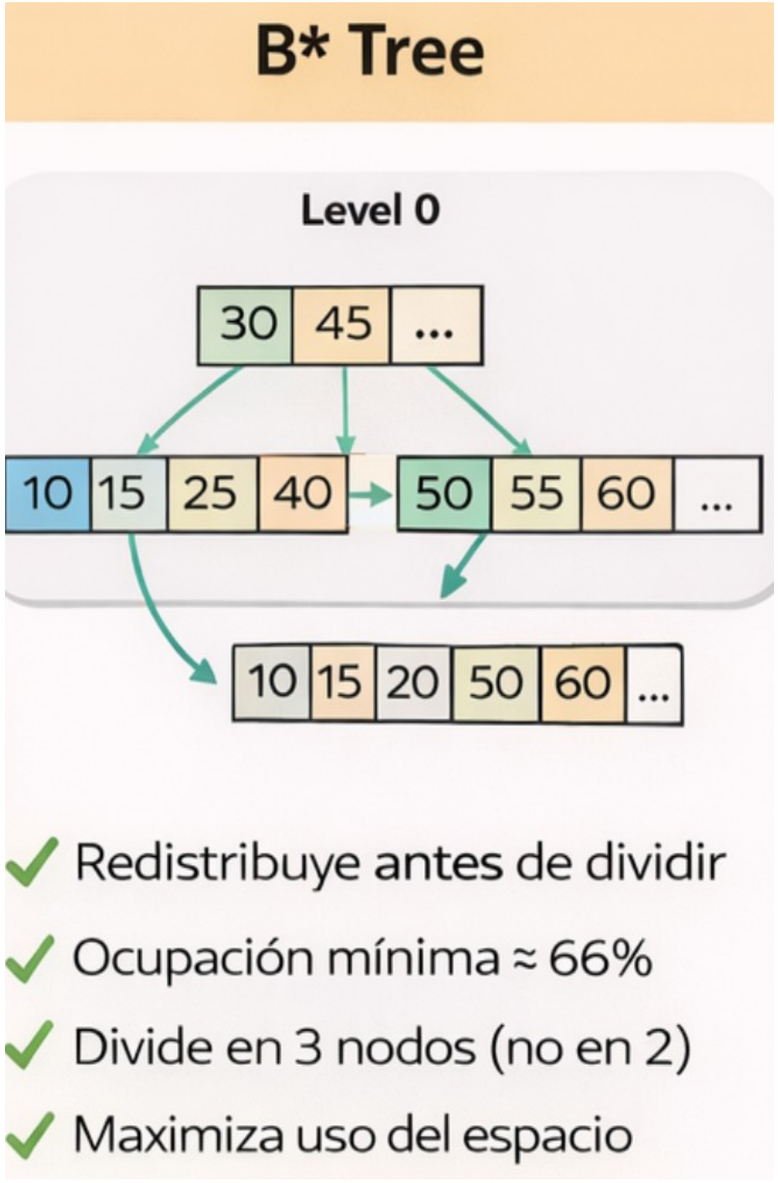
Propiedades estructurales:

Cada nodo tiene como:

máximo **m hijos** y hasta **m−1 claves**

al menos:**⌈(2m − 1) / 3⌉ hijos**

(≈ 2/3 de ocupación)



Características:

Evitar dividir nodos prematuramente

Antes de dividir:

intenta **redistribuir claves con nodos vecinos**

Ocupación mínima:

- B-Tree: ≥ 50%
- B*: ≥ **66% (⅔)**

Más lleno ⇒ mejor uso de páginas

Intenta redistribuir con el hermano

Divide en **3 nodos en lugar de 2**

Estructura:

- mínimo: ⌈(2m − 1) / 3⌉ − 1 claves
- máximo: m − 1 claves

B* exige mayor ocupación (≈ 2/3) que B (1/2)
nodos más llenos
menos altura
menos accesos a disco

Comparación directa A-B vs B vs B+ vs B*

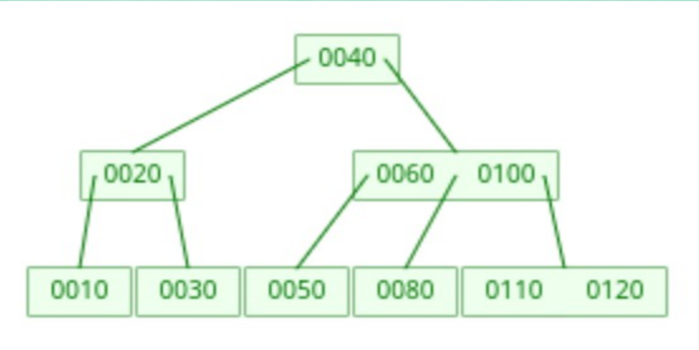
Concepto	Árbol A-B	Árbol B	Árbol B+	Árbol B*
Hijos	$A \le k \le B$	$\lceil m/2 \rceil \le k \le m$	$\lceil m/2 \rceil \le k \le m$	$\lceil (2m-1)/3 \rceil \le k \le m$
Parámetros	A y B libres	definidos por m	definidos por m	definidos por m
Orden	B	m	m	m
Uso	Teórico	Práctico (BD, discos)	Índices en BD, FS modernos	Optimización de almacenamiento
Claves	$k-1$	$k-1$	$k-1$ (internos) / datos en hojas	$k-1$
Datos	—	Internos y hojas	Solo en hojas	Internos y hojas
Hojas enlazadas	—	**No**	**Sí**	**No** (salvo variantes)
Ocupación mínima	—	50%	50%	≈ 66%
Estrategia	—	Split en 2	Split en 2	Redistribución + split en 3

Evolución

Estructura	Autor / referencia
B-Tree	Bayer & McCreight (1972)
B+ Tree	Evolución práctica (sin autor único)
B* Tree	Asociado a Knuth (1973)

Comparación B-Tree vs. B+ Tree vs. B* Tree

B-Tree



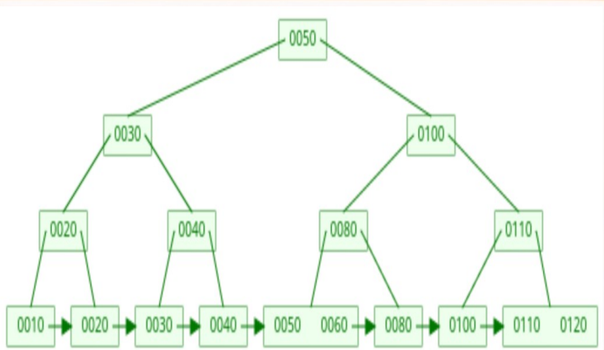
- ✓ Datos en **nodos internos**
- ✗ Hojas no **enlazadas**
- ✓ Los datos pueden estar en interna y hojas
- ✓ 50% ocupación mínima

BD, discos antiguos

- ✓ BD, discos antiguos
- ✓ BD, discos **antiguos**
- ✓

VS

B+ Tree



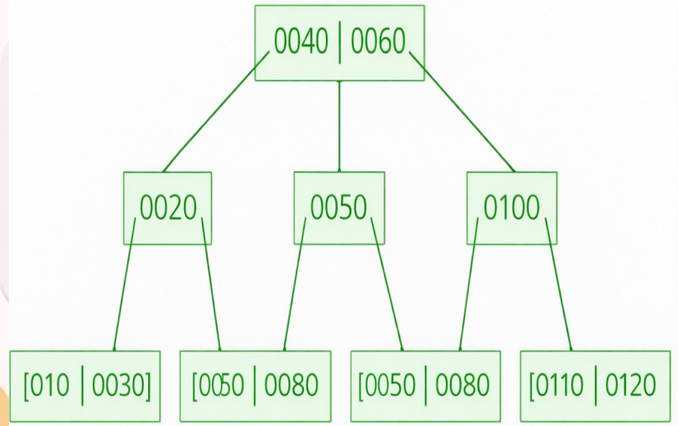
- ✓ Datos **solo** en hojas
- ✓ Hojas **enlazadas**
- ✓ Optimiza búsquedas por rango
- ✓ 50% ocupación mínima

MySQL PostgreSQL SQL Server

- ✓ BD, sistemas modernos
- ✓ BD, discos antiguos
- ✓

VS

B* Tree



- ✓ Redistribuye antes de dividir
- ✓ Ocupación mínima ≈ 66%
- ✓ Divide en 3 nodos (no en 2)
- ✓ Maximiza uso del espacio

Optimización intensiva

- ✓ Optimización **intensiva**
- ✓

TRIE: Árboles Lexicográficos

Anexo

¿Qué es un TRIE?

Estructura de datos de tipo árbol para recuperación de información (del inglés re**TRIE**val). Propuesto en 1959 por Briandais y Fredkin. La búsqueda es análoga a un diccionario: se desciende desde la raíz siguiendo el símbolo buscado hasta la hoja, donde se compara la cadena completa.

Armado del TRIE

1. Insertar todas las claves.

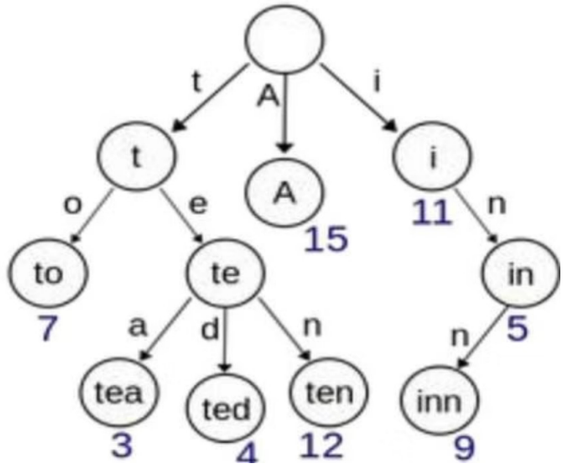
Recorrido en **pre-orden** → obtiene todas las claves.
Recorrido en **post-orden** → orden lexicográfico ascendente.

4. Orden descendente con recorrido inverso.

Implementaciones

Nodo-vector: cada nodo es un vector de punteros para acceso directo a subárboles. Demasiado coste en espacio.

Nodo-lista: cada nodo es una lista enlazada con las raíces de los subárboles (representación primogénito-siguiente hermano). Menos coste en espacio pero más en tiempo de búsqueda



Anexo

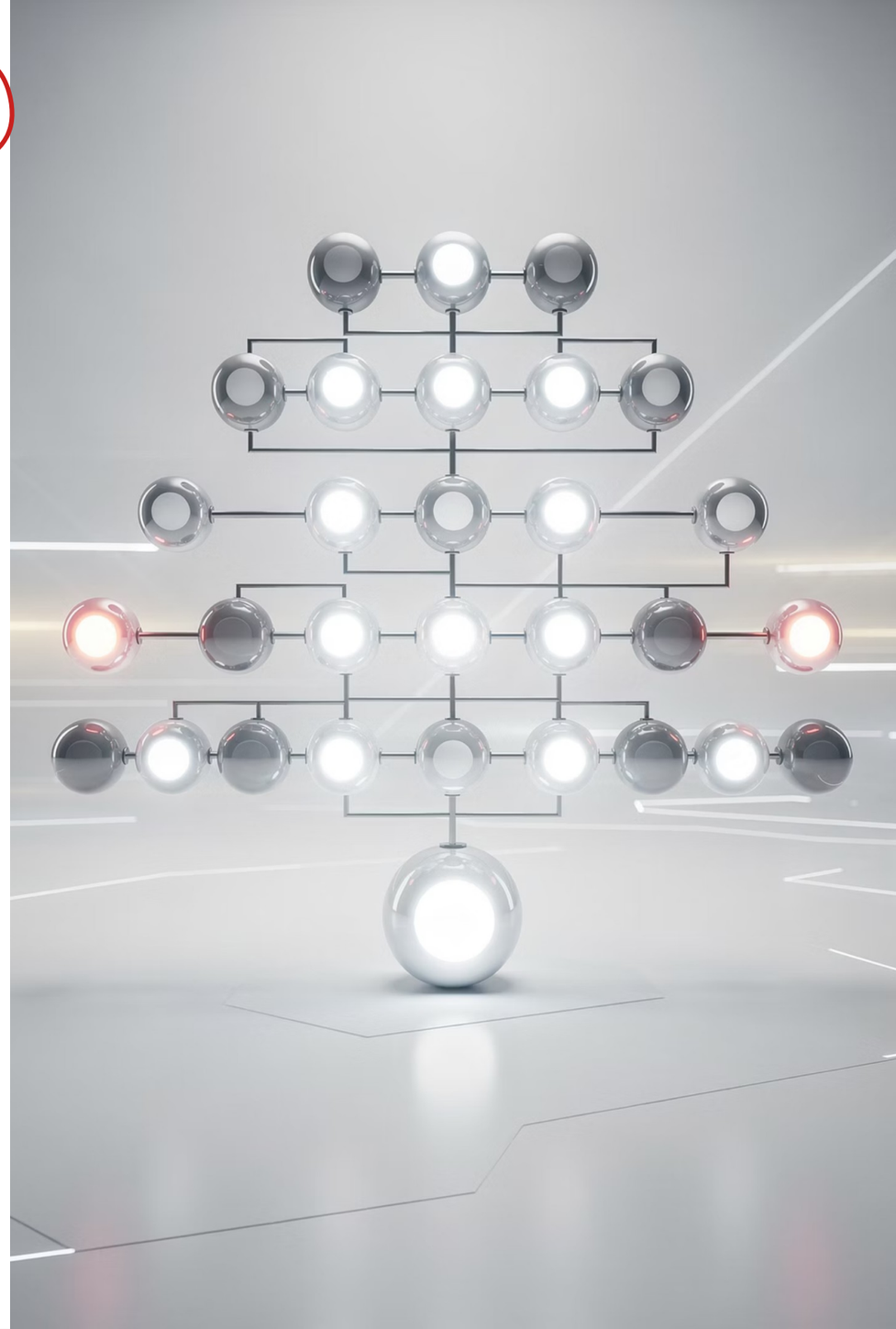
Árbol Patricia

Practical Algorithm To Retrieve Information Coded In Alphanumeric.

Mejora del TRIE que elimina los numerosos punteros nulos. La búsqueda difiere en dos puntos clave:

Testeo Selectivo

Solo se testean los **caracteres indicados en los nodos**, no todos los caracteres de la cadena buscada, reduciendo comparaciones innecesarias.

Comparación Final

La búsqueda **siempre termina** con la comparación completa de la secuencia buscada con la encontrada en la hoja, garantizando corrección.